

PYTHON FORGE

Comprehensive Review & Bounded Roadmap

A prioritized recovery plan for a trustworthy, offline Android beginner-Python MVP—plus an explicit point at which development should stop.

40

Missions audited

Across 8 learning modules

26

Code missions probed

Canonical execution checked

5,541

Dart LOC reviewed

43 authored Dart files

STOP ADDING FEATURES AND LESSONS NOW.

The app has enough scope for a useful beta. Its limiting factor is not feature quantity; it is trust. The execution engine is not Python-compatible, mission success can be awarded after runtime failure, progression is internally inconsistent, and release configuration is not production-ready.

What the product should become

An offline-first Android beginner Python MVP/beta with exactly the current 8 modules and 40 missions. It should teach a bounded introductory path reliably, preserve progress, resume correctly, and run code through an explicit, testable execution contract.

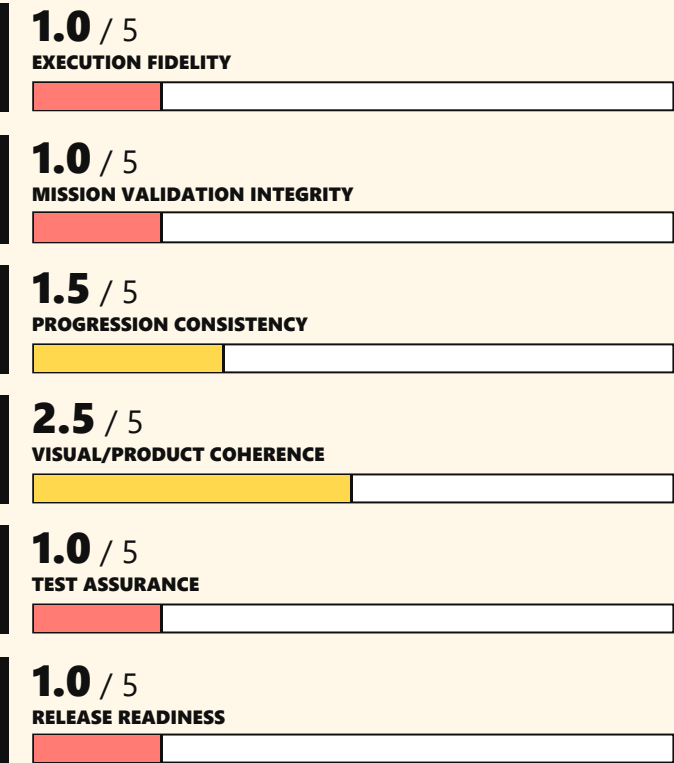
What it should not become yet

Not a social network, cloud IDE, AI tutor, course marketplace, multi-platform classroom suite, or subscription product. Those ideas multiply risk before the basic learning loop is credible.

Recommended remaining investment
3–4 full-time weeks

or 6–8 part-time weeks, followed by a small beta and a two-week defect-only stabilization window.

Readiness scorecard (reviewer judgement, 5 = beta-ready)



Build less. Make the core learning loop true.

What was reviewed

COMPREHENSIVE AUTHORED-CODE AUDIT
GENERATED ARTIFACTS INVENTORIED, NOT TREATED AS PRODUCT LOGIC

43

Dart files

All authored Dart source under
`lib/`

5,541

Dart LOC

Screens, services, models,
engine, theme, widgets

40

Missions

8 modules: 26 code, 8 MCQ, 6
fill-blank

3

Native layers

Android Gradle/Kotlin/Python
bridge truth checked

Inspected in depth

- Runtime and answer validation paths
- Mission screen, code editor behavior, hints, output, rewards
- Curriculum JSON content and schema use
- Progress, XP, levels, locks, resume, streaks, achievements
- Navigation, Riverpod usage, repositories, component systems
- Settings, haptics, font size, reset behavior
- Android manifests, MethodChannel, Python scaffold, signing
- Dependencies, versions, README, tracked structure dumps
- Current tests and Git baseline

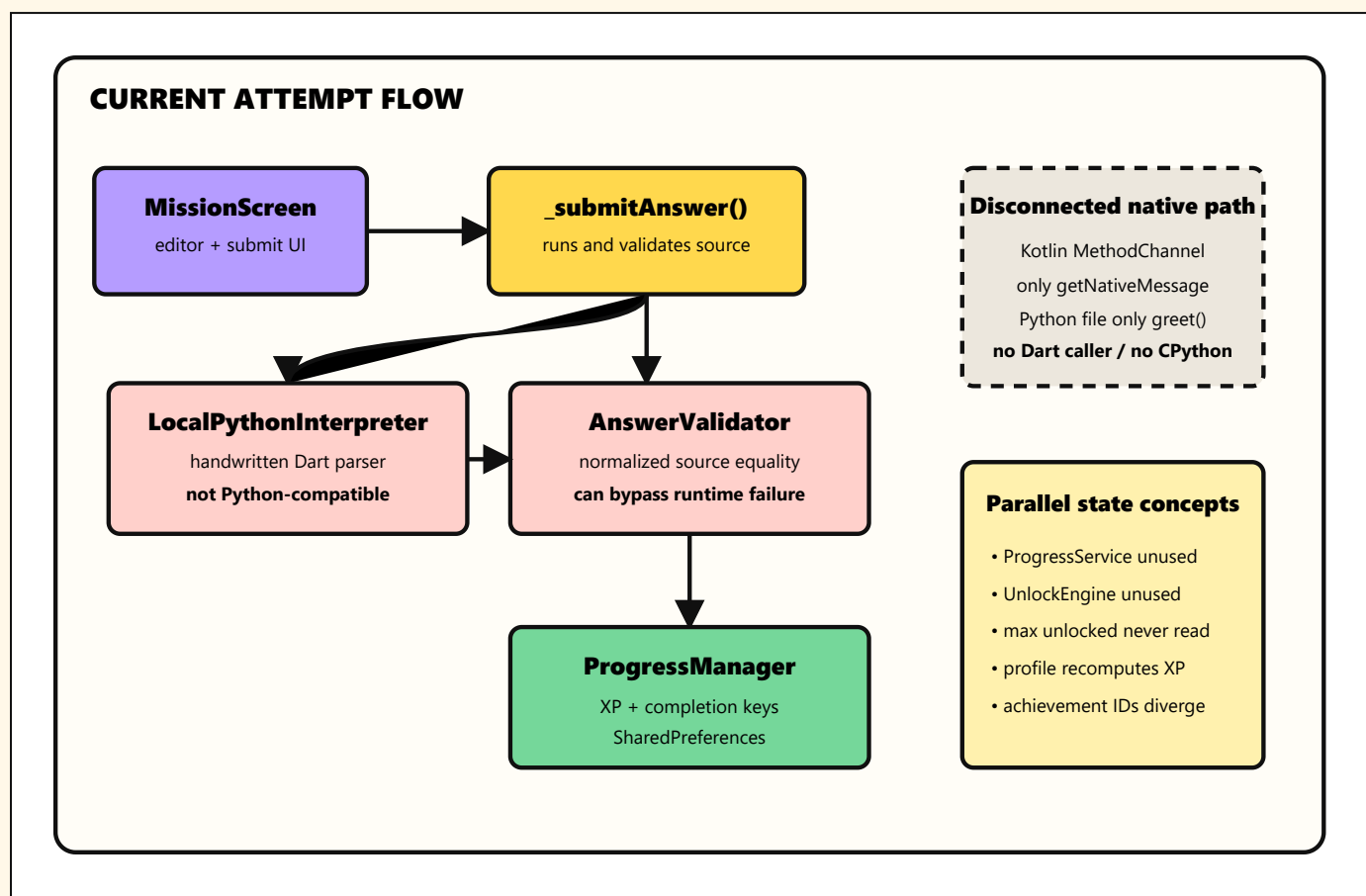
Validation performed

- `flutter analyze --no-pub`: clean
- `flutter test --no-pub`: 1 existing smoke test passed
- Temporary execution probe over all 26 canonical code missions
- Representative beginner expressions tested against the local interpreter
- Baseline Git worktree verified clean before report creation

Important: a clean analyzer proves static conformance, not learning correctness. Generated caches/build outputs and stock platform boilerplate were mapped but not line-reviewed because they are generated, duplicated, or non-actionable.

Primary evidence paths

Concern	Evidence
Execution	<code>lib/core/engine/python_runner.dart</code> , <code>lib/core/engine/answer_validator.dart</code>
Learning loop	<code>lib/features/curriculum/presentation/screens/mission_screen.dart</code> , <code>assets/curriculum/missions.json</code>
Progression	<code>lib/core/progression/progress_manager.dart</code> , achievement and profile services/screens
Native truth	<code>android/app/build.gradle.kts</code> , <code>MainActivity.kt</code> , <code>android/app/src/main/python/main.py</code>
Release	<code>pubspec.yaml</code> , manifests, Gradle signing, <code>README.md</code>



Semantic gap

The engine tokenizes a small language-shaped subset through regular expressions and string splitting. Familiar Python syntax frequently fails or behaves differently.

Validation gap

Canonical source text can be accepted despite an execution error. Alternate correct code can be rejected because equivalence is text-based rather than outcome-based.

State gap

Multiple progress concepts and key formats coexist. Unlock and resume behavior are not driven by one authoritative model.

Execution probe: concrete failures

26 CANONICAL CODE MISSIONS
TEMPORARY PROBE REMOVED AFTER USE

26

Canonical programs

1

Raised runtime error

4

Produced no stdout

1

No-output case expected

Canonical curriculum code already fails the shipped engine

m4_1 • List Mutation produced `NameError: name 'fruits.append("banana")' is not defined`.

Incorrect no-output cases: m3_4 Break Statements, m5_1 Defining Functions, m6_5 Instance Methods. Expected no output: m6_1 Defining Classes.

Representative learner inputs that the current parser mishandles

Ordinary beginner input	Observed structural problem	Learning risk
<code>3 * 4</code> , <code>10 - 4</code> , modulo, floor division, powers	Frequently treated as a name rather than evaluated expression	Correct Python is presented as wrong
<code>"Hello " + "World"</code>	String parsing yields incorrect literal-like output	Teaches false concatenation semantics
<code>print("#1 fan")</code>	Comment stripping removes content after <code>#</code> inside a string	Valid strings can silently disappear
Tuples, <code>input()</code> , f-strings, <code>.upper()</code> , <code>.append()</code> , stepped <code>range</code>	Unsupported or incompletely parsed	Curriculum/runtime contract diverges
Function/method output	Output is routed into a dummy buffer and lost	Working solutions appear to do nothing
Nested <code>break</code>	Control-flow signal does not propagate correctly	Loop behavior is semantically wrong

Why this is P0 rather than “technical debt”

The app’s central promise is that a learner writes Python and receives trustworthy feedback. If valid Python fails, invalid work passes, or output vanishes, the educational contract is broken. UI polish cannot compensate for that.

P0 — Restore product truth

RANKED 01-07
DO BEFORE ANY NEW FEATURE

P0 • RELEASE BLOCKER

- 01 Freeze scope immediately.** Cap the product at 8 modules / 40 missions. Write one sentence of product truth: "Offline Android beginner-Python practice app." Reject all feature/content requests until the stop gate is met.
- 02 Run a 2–3 day execution spike.** Compare an Android embedded CPython route (Chaquopy), a Flutter-oriented packaged runtime candidate (`serious_python`), and an explicitly constrained subset. Test APK/AAB size, startup time, stdout/stderr capture, package boundaries, cancellation feasibility, supported Android APIs, and license obligations.
- 03 Replace the handwritten interpreter as the long-term runtime.** Introduce an `ExecutionPort` and adapter. If a real runtime cannot meet safety/size constraints, rename and document the product as a constrained Python-like exercise environment—never silently claim full Python.
- 04 Make validation fail closed.** A timeout, runtime exception, non-zero execution status, or engine error must always prevent completion and XP. Remove every source-equality path that can override execution failure.
- 05 Replace normalized-source matching with mission contracts.** Code missions should validate successful execution plus explicit expected stdout and optional structured concept checks. MCQ should compare option IDs; fill-blank should compare normalized answer values—not run pseudo-Python.
- 06 Version and validate the curriculum schema.** Add stable IDs, mission type, difficulty, expected output, prerequisites, starter-code policy, accepted-answer strategy, hints, explanation, and schema version. Reject invalid assets in CI and at development startup.
- 07 Correct and re-review all canonical solutions.** Require all 26 code answers to execute with exact declared outcomes. Add representative alternate solutions for each concept so the app rewards Python correctness rather than author formatting.

Higher layers may only pass
after every lower layer passes.



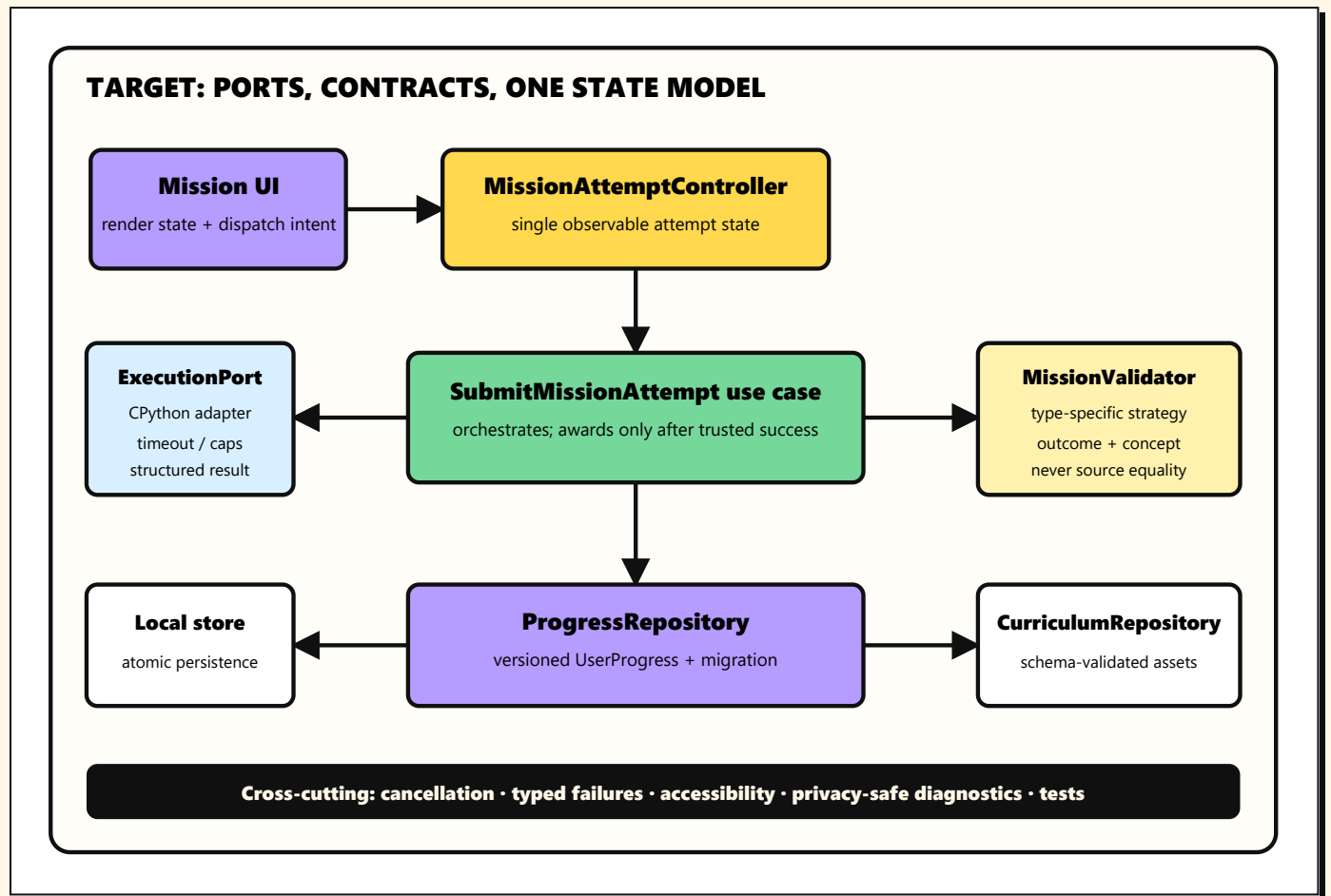
P0 — Make the MVP releasable

RANKED 08–14
STILL BEFORE BETA

- 08 Enforce execution safety.** Define wall-clock timeout, output byte/line cap, recursion/step policy where possible, memory/process boundary strategy, cancellation, deterministic input policy, and a clear “packages are unavailable” rule. Never execute unbounded learner code on the UI isolate.
- 09 Consolidate one progress schema and repository.** Replace mixed `completed_missions`, `${missionId}_completed`, unused service, and stub calculations with a versioned `UserProgress`. Add idempotent migration and deduplication.
- 10 Implement real prerequisites, unlocks, and resume.** Populate prerequisites/difficulty, derive unlocks from completion, read what is written, and make “Resume Learning” select the next incomplete reachable mission—not Module 1.
- 11 Create focused automated assurance.** Add table-driven engine adapter tests, validator failure tests, 40-mission schema tests, canonical/alternate solution tests, progress migration tests, unlock/resume tests, achievement mapping tests, and a small navigation/widget suite. Keep the existing smoke test, but do not mistake it for coverage.
- 12 Add CI quality gates.** On every change: format check, analyzer, targeted tests, curriculum validation, canonical execution contract, and Android release build. Fail if a runtime error can produce completion.
- 13 Establish release identity.** Replace `com.example.python_forge`, choose final display name, align the version source, supply final launcher icon/assets, configure protected release signing, and produce a signed AAB. Never ship debug signing.
- 14 Handle failures as product states.** Show distinct syntax/runtime/timeout/validation/internal errors, preserve learner code, offer actionable guidance, prevent duplicate submissions, and provide a safe retry path. Record only privacy-safe diagnostics if consented.

P0 definition of done

- | | |
|---|---|
| ☐ 26/26 canonical programs execute correctly | ☐ Progress migration/restart/resume verified |
| ☐ Runtime failure can never award completion | ☐ All 40 mission records schema-valid |
| ☐ Alternate valid solutions pass | ☐ Signed release AAB device-tested |



One attempt result

Status, stdout, stderr, error kind, duration, truncation, and optional checks—never ambiguous strings.

One progress authority

XP, completion, unlocks, streak history, achievements, and resume derive from the same versioned state.

One UI state owner

Riverpod controller/notifier is appropriate if retained; widgets render state rather than coordinate services directly.

Runtime decision: make it explicit

SHORT SPIKE, EVIDENCE-BASED CHOICE
NO PREMATURE COMMITMENT

Option	Best case	Main concerns to prove	Recommendation
Chaquopy / embedded CPython on Android	Real Python semantics in the Android-only product boundary	Flutter bridge design, artifact size, API support, cancellation/isolation, licensing, build complexity	SPIKE FIRST Strong Android candidate
serious_python / packaged Flutter runtime	Flutter-oriented integration and possible future platform flexibility	Current maintenance, supported platforms/architectures, runtime constraints, package size, safety model	SPIKE FIRST Compare directly
Documented constrained subset	Small, deterministic, offline learning exercises	Must stop claiming general Python; curriculum and error messages must precisely match supported grammar	FALLBACK Only with honest positioning
Remote sandbox	Server-controlled isolation and broader runtime flexibility	Backend operations, connectivity, privacy, abuse prevention, latency, ongoing cost	DEFER Wrong default for bounded offline MVP

Spike acceptance measurements

- All 26 canonical solutions + alternate cases
- Cold/warm startup time on two devices
- APK/AAB size delta and ABI impact
- stdout/stderr and multiline input contract
- Timeout/cancel behavior for loops and recursion
- File/network/module exposure boundaries
- Crash behavior and lifecycle interruption
- License notices and distribution constraints

Non-negotiable safety stance

"Local" is not automatically "safe." Learner-authored code must not freeze the UI, write arbitrary app data, flood output, access unintended resources, or run without a termination strategy.

Do not promise a perfect sandbox unless the chosen runtime/process boundary can prove it. Instead, state supported behavior, cap resources, restrict exposed APIs, and test adversarially.

Decision rule after Sprint 0

Choose the smallest option that passes semantics, bounded execution, Android release build, and maintainability criteria. If none passes, reduce the runtime claim and curriculum—not safety.

P1 — Repair product consistency

RANKED 15–24
REQUIRED QUALITY BEFORE BROAD LAUNCH

- 15** **Unify the achievement catalog.** Engine IDs (`first_program` , `explorer` , `apprentice` , `scholar` , `persistent`) do not match screen IDs (`first_code` , `streak_3` , `module_1_done` , `module_2_done` , `xp_500`). Create one typed catalog and remove placeholder logic.
- 16** **Store real streak history.** The weekly UI currently infers/fabricates prior active days from a current count. Persist date-based activity, define timezone/day boundaries, and test missed-day behavior.
- 17** **Honor settings everywhere.** Gate haptic calls behind the preference; make editor font changes reactive; scale starter-prefix styling; persist settings through progress reset.
- 18** **Make reset semantics precise.** “Reset progress” should delete progress/XP/streak/achievements only, require confirmation, preserve preferences, and be recoverable until final confirmation. Avoid blanket `SharedPreferences.clear()` .
- 19** **Protect starter code correctly.** Styling is not immutability. Enforce a non-editable prefix/range or model starter code separately; guarantee a separating newline; preserve cursor/selection during updates.
- 20** **Improve teaching feedback.** Add concise explanations after submission, graduated hints, precise error guidance, multiple valid solution styles, and a mastery check at each module boundary.
- 21** **Make difficulty meaningful.** Missions currently default to easy, flattening rewards. Populate calibrated difficulty or remove difficulty-based reward claims until data exists.
- 22** **Perform a complete accessibility pass.** Replace bare `GestureDetector` controls with semantic Material actions where appropriate; add focus/keyboard behavior, labels, tooltips, minimum targets, visible states, contrast checks, and screen-reader ordering.
- 23** **Make layout responsive.** Remove fragile hardcoded widths/heights, test small Android screens and landscape, support large text without clipping, respect safe areas, and allow content to reflow.
- 24** **Align design truth.** Choose the cream/black/yellow neubrutalist system actually used, centralize tokens/components, update documentation, and either bundle declared fonts or use system fallbacks intentionally.

P1 — Simplify engineering ownership

RANKED 25–32
REDUCE FUTURE REGRESSION COST

- 25** **Choose one navigation model.** GoRouter currently defines only `/` and `/mission` while most navigation uses `Navigator`. Standardize, add typed route inputs, and remove unsafe `state.extra as Mission` deep-link behavior.
- 26** **Choose one state-management pattern.** If Riverpod remains, use it for feature controllers/use cases/repositories rather than one mostly dead provider. Otherwise remove it. Avoid service access and mutable orchestration directly in screens.
- 27** **Split large screens by concern.** Refactor `mission_screen.dart` (905 lines), `python_runner.dart` (757), `home_screen.dart` (632), `profile_screen.dart` (455), and `settings_modal.dart` (429) into focused domain/application/presentation units.
- 28** **Consolidate code editor implementations.** Keep one editor, one starter-code policy, one output/error panel, one settings integration, and one test surface.
- 29** **Consolidate the component library.** Reuse a single neubrutalist button/card/section system and theme tokens. Remove parallel visual systems and hardcoded local variants.
- 30** **Make repositories real boundaries.** Use `MissionRepository` and progress repositories behind use cases—or remove abstractions that provide no substitution/testing value. Avoid “architecture decoration.”
- 31** **Establish typed domain failures.** Replace ad hoc strings/exceptions with sealed result types for curriculum load, execution, validation, persistence, and migration; map these to friendly UI states.
- 32** **Centralize version and environment metadata.** Read package metadata instead of maintaining an unused `0.1.0-dev` provider beside `0.5.0+5`. Keep build flavor/signing differences explicit.

Refactor rule

Do not perform a repository-wide rewrite. Refactor vertically around the mission-attempt flow first, preserve behavior with contract tests, then migrate progress and remaining screens. Every abstraction must remove a duplicated truth or create a test seam.

P2 · AFTER BETA BLOCKERS

- 33** **Delete proven dead code** after references/tests confirm it: unused `ProgressService`, `UnlockEngine`, legacy providers/widgets, duplicate editor/components, and unused repository implementations.
- 34** **Remove unused dependencies**, including `logger` if still unreferenced; pin and audit only what is needed.
- 35** **Remove misleading native scaffolding** if the selected runtime does not use `main.py` / current `MethodChannel`. If retained, integrate and document it truthfully.
- 36** **Remove tracked structure dumps** (`lib_structure.txt`, bloated UTF-16 `project_structure.txt`) and generate them only on demand.
- 37** **Rewrite README** with correct clone syntax, supported platform/runtime truth, setup/build/test instructions, architecture summary, limits, and release status.
- 38** **Add privacy-friendly diagnostics only if needed:** local error counters or opt-in crash reporting with disclosure, retention, and no learner source upload by default.
- 39** **Document maintenance ownership:** curriculum change checklist, schema migration policy, release checklist, and dependency update cadence.

P3 · EXPLICITLY OUT OF SCOPE

Do not build before beta evidence

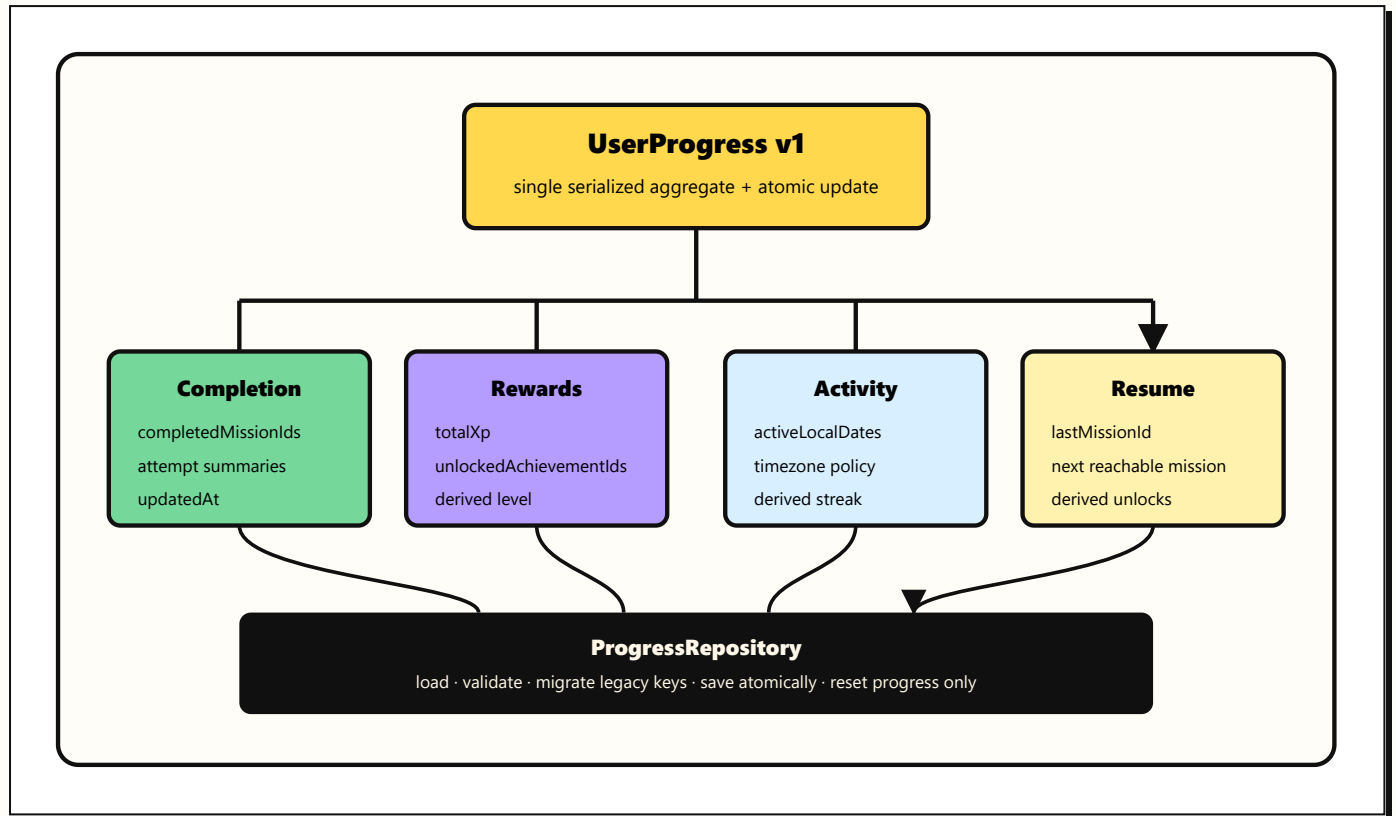
- Authentication and user accounts
- Cloud sync or multi-device state
- AI tutor/chat or generated lessons
- Social feed, comments, friends
- Leaderboards and competitive ranking
- Subscriptions, payments, ads
- Course builder or creator marketplace
- More modules or advanced Python tracks
- Remote execution backend
- Desktop/iOS expansion

- 40** **Only reconsider cloud sync** after repeat users demonstrate cross-device loss is a real blocker.
- 41** **Only reconsider AI tutoring** after deterministic explanations/hints are proven insufficient and privacy/cost can be justified.
- 42** **Only add curriculum** when Module 1–8 completion data and novice interviews identify a specific learning gap—not because the roadmap looks empty.

Scope pressure test: if an idea does not improve execution truth, learning feedback, progress reliability, accessibility, or release safety for the current 40 missions, it is not part of this MVP.

Progression: one durable state

REPLACE KEYS AND RECOMPUTATION
WITH A VERSIONED DOMAIN MODEL



Derived, never independently stored

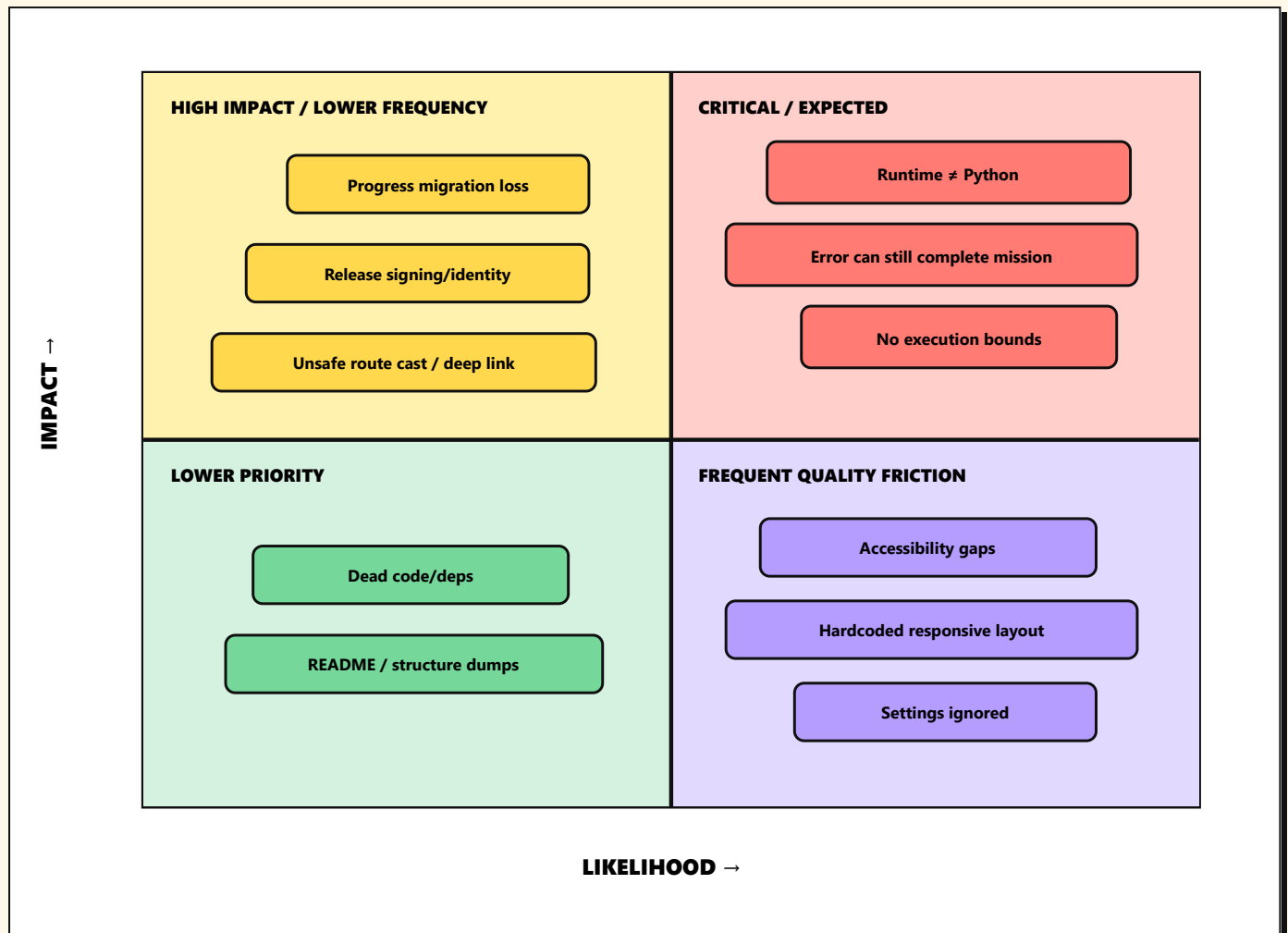
Current level, completion percentage, current/max unlock, streak count, module completion, next mission, and achievement visibility derive from canonical IDs and catalog rules.

Migration invariants

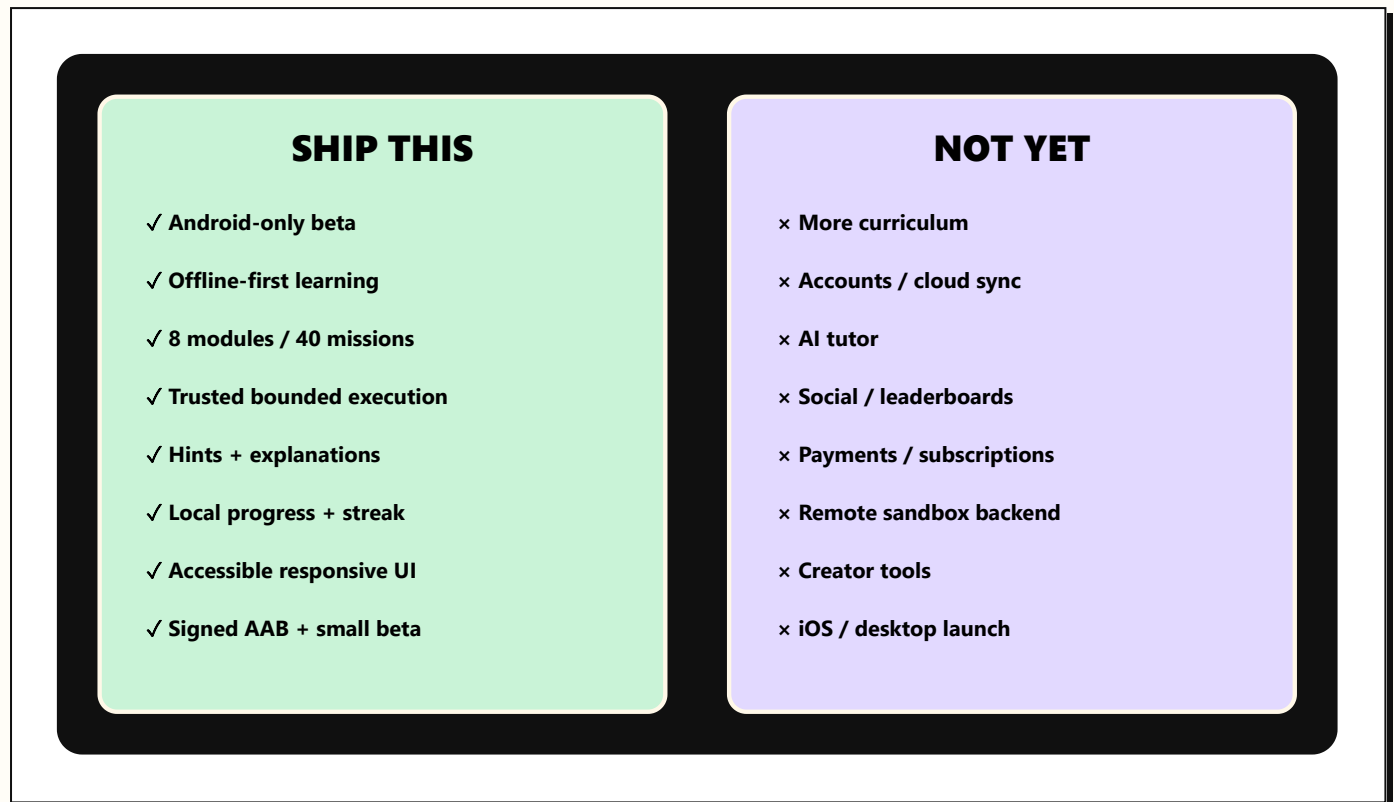
Idempotent; deduplicates legacy/current keys; never loses recognized completion; ignores unknown IDs safely; persists schema version; settings live outside progress and survive reset.

Risk map

SEVERITY × LIKELIHOOD
WORK FROM TOP-RIGHT TOWARD BOTTOM-LEFT



Management implication: the three top-right risks all sit in the same mission-attempt path. Fixing that vertical slice yields more product value than parallel work on profile polish, extra lessons, cloud features, or visual animation.



Target learner

A novice on Android completing a short, guided introduction to Python fundamentals without requiring an account or network connection.

Target outcome

The learner can read and write basic values, conditions, loops, collections, functions, classes, and methods with truthful execution feedback.

Target evidence

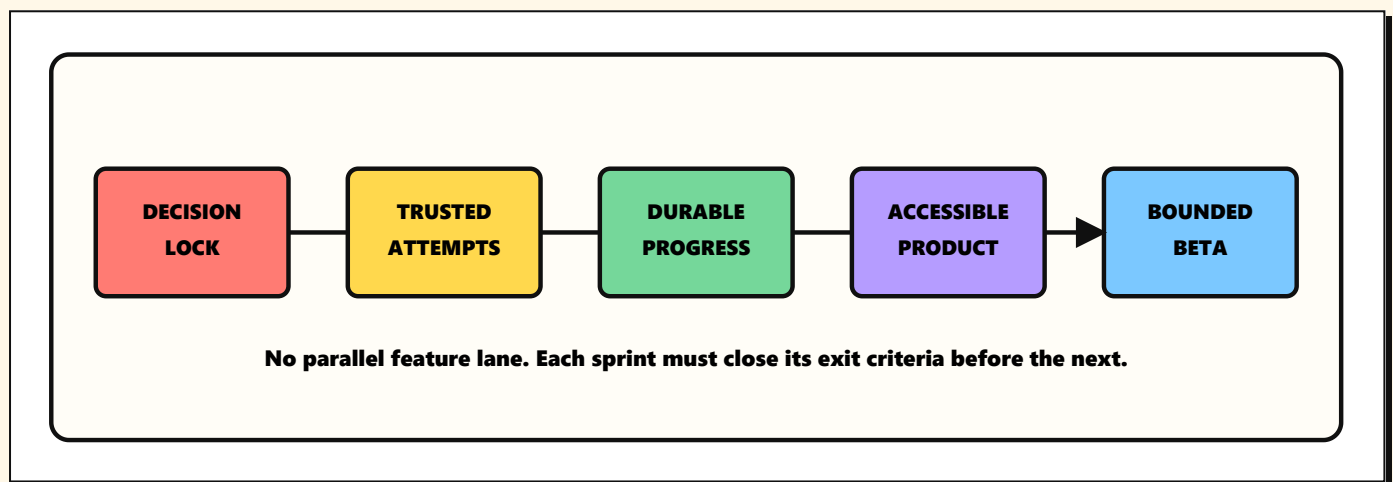
Completion, test, device, accessibility, and novice-usability gates—rather than feature count or visual completeness.

A bounded product is not an abandoned product. It is a product with a testable promise.

Five-sprint finish plan

3–4 FULL-TIME WEEKS
OR 6–8 PART-TIME WEEKS

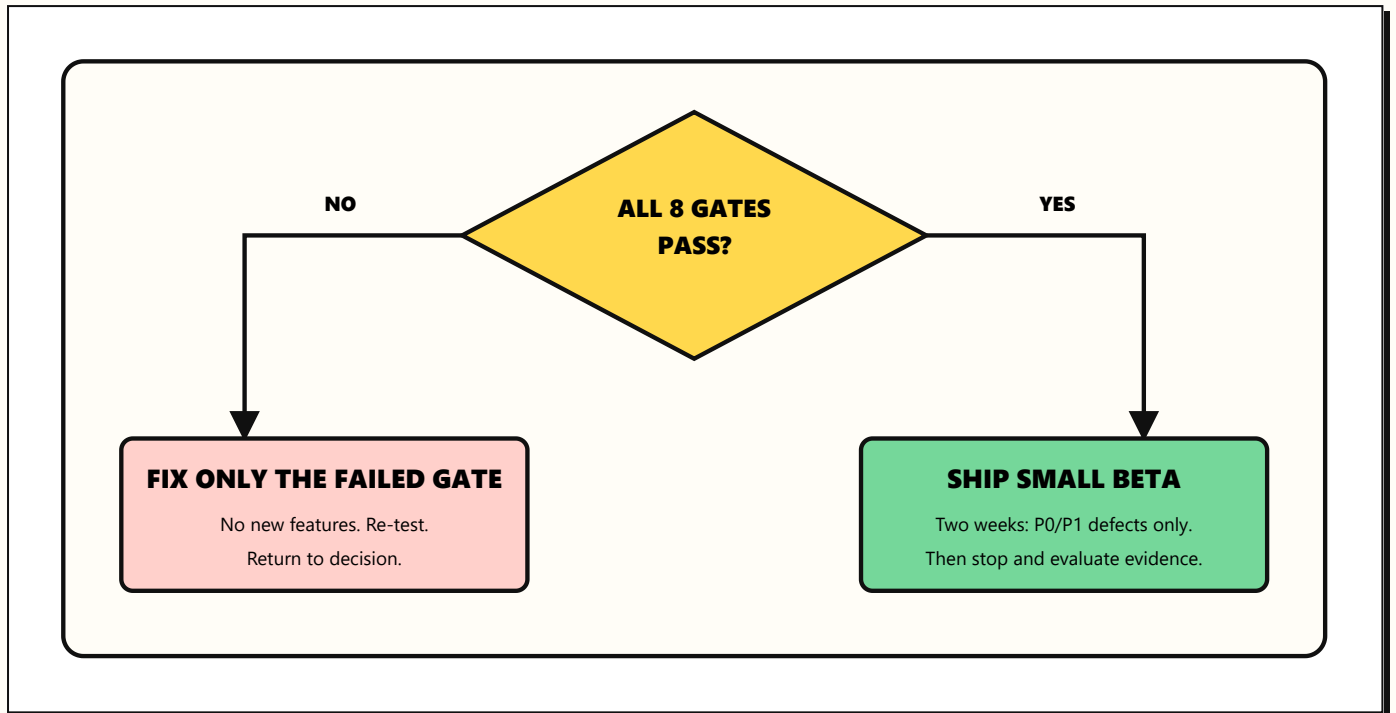
S0 Decide 2–3 days • Freeze scope • Write runtime contract • Spike runtime candidates • Define gates and risks Exit: evidence-based runtime choice.	S1 Trust 5–7 days • ExecutionPort + adapter • Typed results/errors • Timeout/output caps • Validator strategies • Canonical + alternate tests Exit: mission cannot pass on failure.	S2 Persist 4–5 days • Schema v1 + migration • Unlock + resume • Difficulty/rewards • Achievement catalog • Real streak dates Exit: restart-safe learning path.	S3 Polish 4–5 days • Controller/use-case split • One navigation system • Editor/settings fixes • Accessibility • Responsive screens Exit: usable under large text and small screens.	S4 Ship beta 3–5 days • 40-mission content QA • Release identity/signing • Two-device test • README/release checklist • 5-person novice pilot Exit: signed beta passes stop gate.
---	---	--	--	---



Capacity rule: reserve approximately 20% of each sprint for regression fixes and integration. If runtime integration overruns, cut refactoring polish—not correctness, safety, testing, or release validation.

The stop-building gate

FINITE SUCCESS CRITERIA
WHEN EVERY ITEM IS TRUE, SHIP AND STOP



1

Execution

100% of 26 canonical code solutions execute with declared output.

2

Integrity

Runtime failure, timeout, or engine error can never complete a mission.

3

Flexibility

Representative alternate valid solutions pass every taught concept.

4

Progress

Restart, migration, resume, prerequisites, locks, XP, and achievements are correct.

5

Curriculum

All 40 missions are manually reviewed and schema-validated.

6

Quality

Analyzer and focused tests pass in CI; release errors are actionable.

7

Device/accessibility

Signed AAB works on two Android API/device profiles; large-text smoke checks pass.

8

Novice usability

At least 4 of 5 target novices finish Module 1 without blocking help.

Acceptance checklist

USE AS THE IMPLEMENTATION BOARD
BINARY EVIDENCE, NOT "LOOKS DONE"

Execution & validation

- ☐ Runtime choice documented with spike evidence
- ☐ Execution result is structured and typed
- ☐ Timeout, cancellation, output cap implemented
- ☐ No UI-isolate unbounded execution
- ☐ Runtime errors always fail attempts
- ☐ Expected stdout explicit per code mission
- ☐ MCQ/fill strategies are type-specific
- ☐ Source-normalization bypass removed
- ☐ Canonical and alternate suites pass

Curriculum & learning

- ☐ Schema version and validator added
- ☐ All IDs unique and stable
- ☐ Prerequisites populated and acyclic
- ☐ Difficulty/reward model calibrated
- ☐ Starter code cannot be accidentally edited
- ☐ Hints are graduated, not solution dumps
- ☐ Success explanation teaches why
- ☐ All 40 missions content-reviewed
- ☐ Module mastery checks piloted

Progress & product state

- ☐ One versioned progress aggregate
- ☐ Legacy completion deduplicated/migrated
- ☐ Resume selects next reachable mission
- ☐ Unlock rules derive from prerequisites
- ☐ Achievement IDs use one catalog
- ☐ Streak uses persisted local dates
- ☐ Progress reset preserves preferences
- ☐ Haptic/font preferences apply live

Release & usability

- ☐ Final app ID/name/icon/version aligned
- ☐ Release key protected; signed AAB built
- ☐ Two device/API profiles tested
- ☐ Large text and small screen pass
- ☐ Controls have semantic labels/focus
- ☐ Failures preserve code and allow retry
- ☐ README describes actual runtime/limits
- ☐ 4/5 novice Module 1 completion

Evidence rule: attach a test result, schema report, device recording/screenshot, signed artifact, or usability note to every checked item. A merged code change is not by itself evidence that a gate passed.

Detailed finding index

OBSERVED GAPS GROUPED BY SYSTEM
CROSS-CHECK DURING IMPLEMENTATION

Area	Observed evidence	Disposition
Interpreter	Naive splitting, regex expression handling, string-unaware comment stripping, incomplete control flow, dummy function stdout.	REPLACE
Validation	Canonical normalized source can pass despite runtime error; indentation/newlines/comments/quote distinctions erased; expected outputs inferred heuristically.	REDESIGN
Editor	Starter prefix is styled but editable; newline boundary not guaranteed; prefix font hardcoded.	CORRECT
Curriculum schema	No prerequisites and no difficulty fields in JSON; defaults flatten rewards; no explicit expected-output contract.	VERSION
Progress	Two completion-key schemes; max unlock written/not read; stubs coexist with profile recomputation; possible double count.	UNIFY
Resume/locks	All content accessible; home resume opens first module; unused unlock engine.	IMPLEMENT
Achievements	Engine IDs and screen IDs differ; persistent achievement disabled placeholder.	CATALOG
Streak	Weekly history reconstructed from current count rather than persisted dates.	PERSIST
Settings	Haptics ignored by service; editor font not reactive; reset clears preferences.	WIRE
Navigation	GoRouter and Navigator coexist; mission route force-casts extras.	CHOOSE ONE
State/architecture	Riverpod barely used; repository/provider/service abstractions are unused or duplicated; large screen monoliths.	SIMPLIFY
Design	Theme/README style claims conflict with screen-local neubrutalism; referenced fonts not bundled; duplicate visual systems.	ALIGN
Accessibility	GestureDetector-heavy controls, weak semantics/focus/tooltips, hardcoded dimensions and text-size risk.	AUDIT
Android/native	Python file only has greet(); Kotlin channel only native message; no Dart caller; no CPython plugin configured.	DECIDE TRUTH
Release	Example application ID, lowercase label, debug signing for release, inconsistent version source.	HARDEN
Tests	Only one text-render smoke test; no runtime, validation, curriculum, progression, or migration assurance.	ADD GATES
Hygiene	Unused logger; stale structure dumps; native scaffold and widgets/providers may be dead; README malformed/incomplete.	CLEAN

What is already positive

The app has a coherent beginner concept sequence, a distinctive visual direction, a manageable 40-mission scope, offline ambition, clean static analysis, and a functioning Flutter/Android foundation. These strengths justify a focused recovery rather than abandonment or a rewrite.

Finish the promise. Do not expand the product.

Invest only enough to make the current 40 missions truthful, durable, accessible, and releasable. Then run the small beta, fix P0/P1 defects for two weeks, stop development, and decide the future from learner evidence.

NOW

Freeze, spike, repair

Execution → validation → progress → assurance → release.

THEN

Ship, stabilize, stop

Five novice testers, two devices, two weeks defect-only.

The management answer

How much more? Approximately 3–4 focused full-time weeks (6–8 part-time), provided feature scope remains frozen.

Where to stop? At the eight-gate beta threshold on page 17—not when every idea has been implemented.

What happens next? If the beta proves retention and learning value, authorize one evidence-backed increment. If it does not, archive the MVP cleanly without further sunk-cost development.

Runtime research references

1. [Chaquopy official repository](#) — Android Python integration candidate.
2. [serious python official repository](#) and [Flet Python build tooling](#) — Flutter-oriented candidate and runtime build context.
3. [CPython mobile platform documentation \(iOS\)](#) — broader context for mobile embedding constraints; not evidence of Android integration.

Content derived from linked sources was summarized and rephrased for compliance with licensing restrictions. Runtime candidates are recommendations for a compatibility/safety spike, not unverified guarantees of current suitability. Validate current versions, platform support, licenses, and maintenance status at implementation time.

Audit integrity note: This report is advisory. No existing app source, configuration, asset, or test file was modified to produce the review. The standalone PDF is the only intended retained project artifact.